CSCI-6730 OPERATING SYSTEMS

Project #3 – Virtual Memory

HRUTHI MUGGALLA

ID: 811495373

INTRODUCTION

In this project, we implement a Virtual Memory Simulator to understand page tables and page replacement algorithms. It will simulate a 16-bit virtual machine that supports a linear page table structure with up to 256 pages, 256 frames, and 8 processes. This is a simulator of virtual-to-physical address translation that, will handle page faults, free frame, and dirty frame lists in both physical memory and the swap disk. When all the frames are full, page replacement will be done with algorithms such as: FIFO, LRU, CLOCK, and CLOCK-PRO.

PART – 1: PAGE TABLE

The page table maps virtual pages to physical frames. A physical frame number, a valid bit, and a dirty bit make up each page table entry.

The mapping of physical frames to the corresponding virtual pages and process IDs is helped by the inverted page table: This structure plays an important role during the swapping process because it helps the system identify which virtual page and process correspond to a given physical frame.

The three main functions in pagetable.c file are:

1. **int MMU(int pid, int virtAddr, char reqType, bool *hit)**

   The Memory Management Unit (MMU) actively converts virtual memory addresses into physical memory addresses while guaranteeing efficient management of data. It translates addresses and checks if the requested page is valid in memory. If the page is not found in the page table, a page fault is triggered and the page fault handler is invoked.

The function performs following tasks:

- It extracts the VPN and offset from the given virtual address. This is done by right-shifting the virtual address by 8 bits and masking 0xFF to get the upper 8 bits.

```
int PFN, physicalAddr;
int VPN = (virtAddr >> 8) & 0xFF;
int offset = virtAddr & 0xFF;
```

- It calls get_PFN(pid, VPN, reqType) to retrieve the corresponding PFN of the extracted VPN. If the value returned for PFN is greater than or equal to 0, this then means that the page is in memory; hence, it increments the hitCount and sets the hit pointer to true, meaning the page requested is already loaded in memory.

```
PFN = get_PFN(pid, VPN, reqType);
if (PFN >= 0) {
    stats.hitCount++;
    *hit = true;
} else {
    stats.missCount++;
    *hit = false;
    PFN = pagefault_handler(pid, VPN,
    reqType);
}
```

- Once it has the PFN - from the page table, in case of a hit or from the page fault handler in case of a miss, the physical address is computed by left-shifting the PFN by 8 bits to align with the page size, adding the offset, and returns the computed physical address.

```
physicalAddr = (PFN << 8) + offset;
return physicalAddr;
```

2. **int get_PFN(int pid, int VPN, char reqType)**

   get_PFN returns the Physical Frame Number corresponding to the given Virtual Page Number for a process. This function checks for page entry validity, and on write request, it marks the page as dirty and updates the respective data structure for LRU and CLOCK page replacement algorithms.

```
if (pte.valid) {
    if (reqType == 'W') {
        pte.dirty = true;
        write_PTE(pid, VPN, pte);
    }

    if (replacementPolicy == LRU) update_lru(pte.
    PFN);
    else if (replacementPolicy == CLOCK)
    pageReferenced[pte.PFN] = 1;

    return pte.PFN;
}
return -1;
```

3. **int pagefault_handler(int pid, int VPN, char reqType)**

   - The function invokes get_freeframe() to attempt to acquire a free physical frame. If such a frame is free-if get_freeframe() returns a proper (i.e., non-null) frame number-the function will load the requested page onto it.

- If no frame is free, i.e, PFN < 0, the function performs the page replacement procedure. It calls find_replacement( ) in order to select a victim page to be evicted, according to the active page replacement policy.

- The IPTE of the victim is retrieved by using read_IPTE(PFN). The IPTE contains the pid and the VPN of the victim page.

- If the victim page is dirty, meaning that it has been modified and hence needs to be written back to disk-the function calls swap_out() to write the victim page to the swap disk.

- After having the dirty victim handled, it sets the validity of the victim to false, which indicates that this page no longer is valid in memory. Then it calls write_PTE() to write the updated PTE back to the page table.

- The Inverted Page Table Entry is filled in with the location of the new page in the physical memory. The IPTE is written using write_IPTE(). This function will update the relevant data structures depending on the current page replacement policy.

```
int pagefault_handler(int pid, int VPN, char reqType) {
    int PFN;
    PFN = get_freeframe();

    if (PFN < 0) {
        PFN = find_replacement();
        IPTE victim = read_IPTE(PFN);
        PTE victimPTE = read_PTE(victim.pid, victim.
        VPN);
        if (victimPTE.dirty) {
            swap_out(victim.pid, victim.VPN, PFN);
        }
        victimPTE.valid = false;
        write_PTE(victim.pid, victim.VPN, victimPTE);
    }
    swap_in(pid, VPN, PFN);
```

PART – 2: PAGE REPLACEMENT ALGORITHMS

The virtual memory management page replacement algorithms decide which memory page is to be swapped out (replaced) when a page fault occurs and the physical memory is full. Here, we will implement the FIFO, LRU, CLOCK, and CLOCK_PRO page replacement algorithms

a. **First In First Out (FIFO):**
   The queue of frames will be maintained using a linked list to implement the FIFO page replacement algorithm. As per the principle, the oldest frame, i.e., the head of the queue, is always chosen to get replaced.

   There are three functions which are implementing FIFO:
   1. void initialize_fifo()
      Here, it initializes the FIFO queue used for tracking the order of frames. fifoQueueHead is a pointer to the head of this linked list. This queue will keep track of the order of the frames in which they were added.

   2. int fifo()
      Retrieves the frame number of the head node, victimFrame, which is the oldest frame. The fifoQueueHead should be updated to the next node in the list by calling list_remove_head(); removes the oldest frame from the queue.

```
int fifo() {
    if (fifoQueueHead == NULL) {
        fprintf(stderr, "FIFO queue is
        empty! No frames to replace.\n");
        exit(EXIT_FAILURE);
    }

    int victimFrame = fifoQueueHead->data;
    fifoQueueHead = list_remove_head
    (fifoQueueHead);
    return victimFrame;
}
```

3. void add_to_fifo()
   This function adds a frame to the FIFO queue when a new page is loaded into memory. It calls list_insert_tail() to add a new node containing frameNumber to the end of the linked list and updates fifoQueueHead.

```
void add_to_fifo(int frameNumber) {
    fifoQueueHead = list_insert_tail
    (fifoQueueHead, frameNumber);
}
```

Output of vm file and vm_reference using FIFO algorithm:

```
[hm92170@csci-odin OS_Proj3]$ ./vm 3 1 example_inputs/example.txt
# PAGES: 8, # FRAMES: 3, # PROCS: 1, Replacement Policy: 1 - FIFO
[pid 0, W] 0x0700 --> 0x0000: A [miss]
[pid 0, W] 0x0000 --> 0x0100: B [miss]
[pid 0, W] 0x0100 --> 0x0200: C [miss]
[pid 0, W] 0x0200 --> 0x0000: D [miss]
[pid 0, W] 0x0000 --> 0x0100: E [hit]
[pid 0, W] 0x0300 --> 0x0100: F [miss]
[pid 0, R] 0x0000 --> 0x0200: E [miss]
[pid 0, W] 0x0400 --> 0x0000: G [miss]
[pid 0, R] 0x0200 --> 0x0100: D [miss]
[pid 0, R] 0x0300 --> 0x0200: F [miss]
[pid 0, W] 0x0000 --> 0x0000: H [miss]
[pid 0, W] 0x0300 --> 0x0200: I [hit]
[pid 0, R] 0x0200 --> 0x0100: D [hit]
[pid 0, R] 0x0100 --> 0x0100: C [miss]
[pid 0, R] 0x0200 --> 0x0200: D [miss]
[pid 0, R] 0x0000 --> 0x0000: H [hit]
[pid 0, R] 0x0100 --> 0x0100: C [hit]
[pid 0, R] 0x0700 --> 0x0000: A [miss]
[pid 0, R] 0x0000 --> 0x0100: H [miss]
[pid 0, R] 0x0100 --> 0x0200: C [miss]
==============================
Request: 20
Page Hit: 5 (25.00%)
Page Miss: 15 (75.00%)
Swap Read: 15
Swap Write: 8
[hm92170@csci-odin OS_Proj3]$
```

```
[hm92170@csci-odin OS_Proj3]$ ./vm_reference 3 1 example_inputs/example.tx
t
# PAGES: 8, # FRAMES: 3, # PROCS: 1, Replacement Policy: 1 - FIFO
[pid 0, W] 0x0700 --> 0x0000: A [miss]
[pid 0, W] 0x0000 --> 0x0100: B [miss]
[pid 0, W] 0x0100 --> 0x0200: C [miss]
[pid 0, W] 0x0200 --> 0x0000: D [miss]
[pid 0, W] 0x0000 --> 0x0100: E [hit]
[pid 0, W] 0x0300 --> 0x0100: F [miss]
[pid 0, R] 0x0000 --> 0x0200: E [miss]
[pid 0, W] 0x0400 --> 0x0000: G [miss]
[pid 0, R] 0x0200 --> 0x0100: D [miss]
[pid 0, R] 0x0300 --> 0x0200: F [miss]
[pid 0, W] 0x0000 --> 0x0000: H [miss]
[pid 0, W] 0x0300 --> 0x0200: I [hit]
[pid 0, R] 0x0200 --> 0x0100: D [hit]
[pid 0, R] 0x0100 --> 0x0100: C [miss]
[pid 0, R] 0x0200 --> 0x0200: D [miss]
[pid 0, R] 0x0000 --> 0x0000: H [hit]
[pid 0, R] 0x0100 --> 0x0100: C [hit]
[pid 0, R] 0x0700 --> 0x0000: A [miss]
[pid 0, R] 0x0000 --> 0x0100: H [miss]
[pid 0, R] 0x0100 --> 0x0200: C [miss]
==============================
Request: 20
Page Hit: 5 (25.00%)
Page Miss: 15 (75.00%)
Swap Read: 15
Swap Write: 8
```

b. Least Recently Used (LRU):

We used one linked list, lruPageList, in order to maintain the usage sequence of pages, and used one array, lruPageTracker, to keep track of whether a frame was currently under tracking. It follows the LRU principle: the page that has not been used for the longest time is evicted.

The LRU algorithm mainly has four key functionalities-page tracking, victim selection, usage updates, and efficient search-which have been implemented in the following functions:

1. void initialize_lru()

   The main purpose of this function is to initialize the data structures used for tracking LRU.

```
void initialize_lru() {
    int i;
    lruPageList = NULL;
    for (i = 0; i < MAX_PAGES; i++) {
        lruPageTracker[i] = 0;

    }
}
```

2. int lru()

   This function takes the head of the list-in other words, the least recently used page-victimFrame. It changes the value in the lruPageTracker array to 0 to denote that it is no longer being tracked and returns the victim frame number for replacement.

```
int lru() {
    if (lruPageList == NULL) {
        fprintf(stderr, "LRU list is
        empty! No frames to replace.\n");
        exit(EXIT_FAILURE);
    }
    int victimFrame = lruPageList->data;
    lruPageList = list_remove_head
    (lruPageList);
    lruPageTracker[victimFrame] = 0;
    return victimFrame;
}
```

3. void update_lru(int frameNumber):
   It uses list_remove() to delete the node in the linked list if the node with frameNumber already existed, hence not creating duplicate entries.

```
void update_lru(int frameNumber) {
    lruPageList = list_remove
    (lruPageList, frameNumber);
    lruPageList = list_insert_tail
    (lruPageList, frameNumber);

    lruPageTracker[frameNumber] = 1;
}
```

Output of vm and vm_reference using LRU algorithm:

```
[hm92170@csci-odin OS_Proj3]$ ./vm 3 2 example_inputs/example.txt
# PAGES: 8, # FRAMES: 3, # PROCS: 1, Replacement Policy: 2 - LRU
[pid 0, W] 0x0700 --> 0x0000: A [miss]
[pid 0, W] 0x0000 --> 0x0100: B [miss]
[pid 0, W] 0x0100 --> 0x0200: C [miss]
[pid 0, W] 0x0200 --> 0x0000: D [miss]
[pid 0, W] 0x0000 --> 0x0100: E [hit]
[pid 0, W] 0x0300 --> 0x0200: F [miss]
[pid 0, R] 0x0000 --> 0x0100: E [hit]
[pid 0, W] 0x0400 --> 0x0000: G [miss]
[pid 0, R] 0x0200 --> 0x0200: D [miss]
[pid 0, R] 0x0300 --> 0x0100: F [miss]
[pid 0, W] 0x0000 --> 0x0000: H [miss]
[pid 0, W] 0x0300 --> 0x0100: I [hit]
[pid 0, R] 0x0200 --> 0x0200: D [hit]
[pid 0, R] 0x0100 --> 0x0000: C [miss]
[pid 0, R] 0x0200 --> 0x0200: D [hit]
[pid 0, R] 0x0000 --> 0x0100: H [miss]
[pid 0, R] 0x0100 --> 0x0000: C [hit]
[pid 0, R] 0x0700 --> 0x0200: A [miss]
[pid 0, R] 0x0000 --> 0x0100: H [hit]
[pid 0, R] 0x0100 --> 0x0000: C [hit]
==================================
Request: 20
Page Hit: 8 (40.00%)
Page Miss: 12 (60.00%)
Swap Read: 12
Swap Write: 8
[hm92170@csci-odin OS_Proj3]$
```

```
[hm92170@csci-odin OS_Proj3]$ ./vm_reference 3 2 example_inputs/example.tx
t
# PAGES: 8, # FRAMES: 3, # PROCS: 1, Replacement Policy: 2 - LRU
[pid 0, W] 0x0700 --> 0x0000: A [miss]
[pid 0, W] 0x0000 --> 0x0100: B [miss]
[pid 0, W] 0x0100 --> 0x0200: C [miss]
[pid 0, W] 0x0200 --> 0x0000: D [miss]
[pid 0, W] 0x0000 --> 0x0100: E [hit]
[pid 0, W] 0x0300 --> 0x0200: F [miss]
[pid 0, R] 0x0000 --> 0x0100: E [hit]
[pid 0, W] 0x0400 --> 0x0000: G [miss]
[pid 0, R] 0x0200 --> 0x0200: D [miss]
[pid 0, R] 0x0300 --> 0x0100: F [miss]
[pid 0, W] 0x0000 --> 0x0000: H [miss]
[pid 0, W] 0x0300 --> 0x0100: I [hit]
[pid 0, R] 0x0200 --> 0x0200: D [hit]
[pid 0, R] 0x0100 --> 0x0000: C [miss]
[pid 0, R] 0x0200 --> 0x0200: D [hit]
[pid 0, R] 0x0000 --> 0x0100: H [miss]
[pid 0, R] 0x0100 --> 0x0000: C [hit]
[pid 0, R] 0x0700 --> 0x0200: A [miss]
[pid 0, R] 0x0000 --> 0x0100: H [hit]
[pid 0, R] 0x0100 --> 0x0000: C [hit]
==================================
Request: 20
Page Hit: 8 (40.00%)
Page Miss: 12 (60.00%)
Swap Read: 12
Swap Write: 8
```

c. CLOCK page replacement algorithm:

The Clock page replacement algorithm is the approximation that can be implemented to LRU. Based on a circular structure or conceptual clock, the algorithm keeps track of the usage of the pages and selects a page to replace when a page fault occurs.

1. void initialize_clock()

It sets handPosition to 0. This variable represents the position of the "clock hand," which points to the current frame being considered for eviction. Also, this initializes pageReferenced[i] to 0 for each frame, indicating no pages have been referenced yet. It sets memoryFrames[i] to -1 indicating all frames are initially empty.

```
void initialize_clock() {
    int i;
    handPosition = 0;
    for (i = 0; i < MAX_FRAMES; i++) {
        pageReferenced[i] = 0;
        pageSecondChance[i] = 0;
        memoryFrames[i] = -1;
    }
}
```

2. int clock()
   - A loop continually scans the frames of the circular clock. The page in the current frame has not recently been accessed if pageReferenced[handPosition] is 0. It is selected as the replacement victim. From memoryFrames[handPosition] take the physical frame number victimPFN and place - into memoryFrames[handPosition] (this now makes that frame empty).
   - Reset pageReferenced[handPosition] to 0 - that is, it is no longer referenced - and increment the handPosition to point to the next frame in the circular list.
   - If pageReferenced[handPosition] == 1 then reset the referenced bit - pageReferenced[handPosition] = 0 - and move the hand to the next position. Repeat this process until an unreferenced frame is found.

```
int clock() {
    while (1) {
        if (pageReferenced[handPosition] == 0) {
            int victimPFN = memoryFrames[handPosition];
            memoryFrames[handPosition] = -1;
            pageReferenced[handPosition] = 0;
            handPosition = (handPosition + 1) %
            MAX_FRAMES;
            return victimPFN;
        }
        pageReferenced[handPosition] = 0;
        handPosition = (handPosition + 1) %
        MAX_FRAMES;
    }
}
```

Output for CLOCK algorithm:

```
[hm92170@csci-odin OS_Proj3]$ ./vm 3 3 example_inputs/example.txt
# PAGES: 8, # FRAMES: 3, # PROCS: 1, Replacement Policy: 3 - CLOCK
[pid 0, W] 0x0700 --> 0x0000: A [miss]
[pid 0, W] 0x0000 --> 0x0100: B [miss]
[pid 0, W] 0x0100 --> 0x0200: C [miss]
[pid 0, W] 0x0200 --> 0x0000: D [miss]
[pid 0, W] 0x0000 --> 0x0100: E [hit]
[pid 0, W] 0x0300 --> 0x0000: F [miss]
[pid 0, R] 0x0000 --> 0x0100: E [hit]
[pid 0, W] 0x0400 --> 0x0000: G [miss]
[pid 0, R] 0x0200 --> 0x0000: D [miss]
[pid 0, R] 0x0300 --> 0x0000: F [miss]
[pid 0, W] 0x0000 --> 0x0100: H [hit]
[pid 0, W] 0x0300 --> 0x0000: I [hit]
[pid 0, R] 0x0200 --> 0x0000: D [miss]
[pid 0, R] 0x0100 --> 0x0200: C [hit]
[pid 0, R] 0x0200 --> 0x0000: D [hit]
[pid 0, R] 0x0000 --> 0x0100: H [hit]
[pid 0, R] 0x0100 --> 0x0200: C [hit]
[pid 0, R] 0x0700 --> 0x0000: A [miss]
[pid 0, R] 0x0000 --> 0x0100: H [hit]
[pid 0, R] 0x0100 --> 0x0200: C [hit]
=====================================
Request: 20
Page Hit: 10 (50.00%)
Page Miss: 10 (50.00%)
Swap Read: 10
Swap Write: 5
```

d. CLOCK_PRO page replacement algorithm:

The improved algorithm of Clock, "clock_pro," is a variation of the basic algorithm that introduces a "second chance" mechanism and adds an additional layer of protection via pageSecondChance. This smart strategy prevents any frame from being ejected too early by obtaining more refined policies to replace.

Clock_pro uses a notion of Cold Hand and a Hot Hand to keep track of the memory pages. It observes the page recency and the frequency of use, classifying the pages into hot (heavy in usage) and cold pages (low usage).

A frame is evicted if both:

- pageReferenced[handPosition] == 0: The page has not been accessed recently.
- pageSecondChance[handPosition] == 0: The page has already used its "second chance."

```
if (pageReferenced[handPosition] == 0 && pageSecondChance
[handPosition] == 0) {
    int victimPFN = memoryFrames[handPosition];
    memoryFrames[handPosition] = -1;
    pageReferenced[handPosition] = 0;
    pageSecondChance[handPosition] = 0;
    handPosition = (handPosition + 1) % MAX_FRAMES;
    return victimPFN;
}
```

If it's recently accessed and hasn't had its second chance, that is, pageReferenced == 1 and pageSecondChance == 0 then it gets its second chance, pageSecondChance[handPosition] = 1.

```
if (pageReferenced[handPosition] == 1 && pageSecondChance
[handPosition] == 0) {
    pageSecondChance[handPosition] = 1;
}
```

If it's on its second chance -pageSecondChance == 1- then turn off both pageReferenced and pageSecondChance to 0, making it no longer protected.

```
else if (pageSecondChance[handPosition] == 1) {
    pageReferenced[handPosition] = 0;
    pageSecondChance[handPosition] = 0;
}
```

Every time a frame has been processed, that is, after the aforementioned operations, move handPosition to the next frame using modulo arithmetic.

Output using CLOCK_PRO algorithm:

```
[hm92170@csci-odin OS_Proj3]$ ./vm 3 4 example_inputs/example.txt
# PAGES: 8, # FRAMES: 3, # PROCS: 1, Replacement Policy: 4 - CLOCK_PRO
[pid 0, W] 0x0700 --> 0x0000: A [miss]
[pid 0, W] 0x0000 --> 0x0100: B [miss]
[pid 0, W] 0x0100 --> 0x0200: C [miss]
[pid 0, W] 0x0200 --> 0x0000: D [miss]
[pid 0, W] 0x0000 --> 0x0100: E [hit]
[pid 0, W] 0x0300 --> 0x0000: F [miss]
[pid 0, R] 0x0000 --> 0x0100: E [hit]
[pid 0, W] 0x0400 --> 0x0000: G [miss]
[pid 0, R] 0x0200 --> 0x0000: D [miss]
[pid 0, R] 0x0300 --> 0x0000: F [miss]
[pid 0, W] 0x0000 --> 0x0100: H [hit]
[pid 0, W] 0x0300 --> 0x0000: I [hit]
[pid 0, R] 0x0200 --> 0x0000: D [miss]
[pid 0, R] 0x0100 --> 0x0200: C [hit]
[pid 0, R] 0x0200 --> 0x0000: D [hit]
[pid 0, R] 0x0000 --> 0x0100: H [hit]
[pid 0, R] 0x0100 --> 0x0200: C [hit]
[pid 0, R] 0x0700 --> 0x0000: A [miss]
[pid 0, R] 0x0000 --> 0x0100: H [hit]
[pid 0, R] 0x0100 --> 0x0200: C [hit]
=====================================
Request: 20
Page Hit: 10 (50.00%)
Page Miss: 10 (50.00%)
Swap Read: 10
Swap Write: 5
```